Shelf Voice — 02 Technical Requirements and Architecture

Oct 27, 2026 · @Aradhya Chaudhary

Status and open decisions

Version 1.0, approved by the owner on Oct 27. All seven stack decisions plus repo setup are final; changes from here go through this table. Percentages are confidence that the option fits Shelf Voice best, not market share.

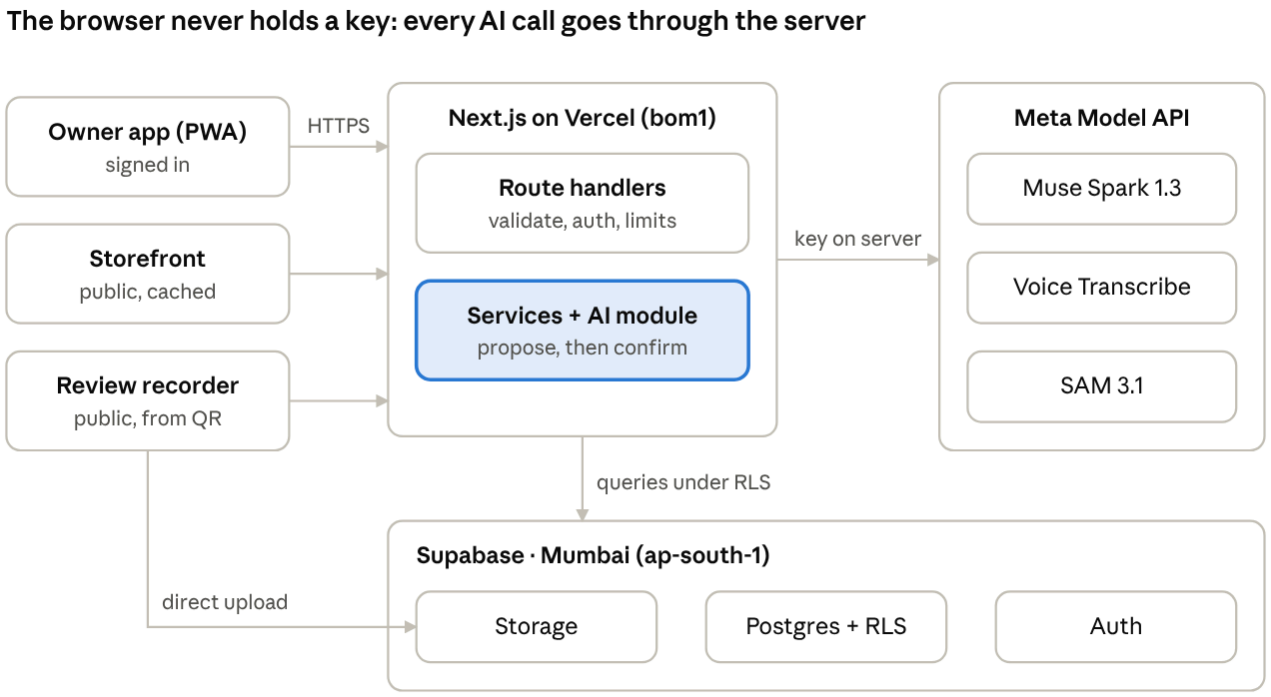
#	Decision	Options (recommendation %)	Why the top option	Owner's choice
T1	Database and backend platform	Supabase Postgres, Mumbai region (75%); Firebase Firestore (15%); Neon Postgres + own auth (10%)	Relational data (products, sales, reviews) fits Postgres; Row Level Security isolates each shop in the database; Mumbai region keeps latency low	Supabase Postgres, Mumbai
T2	App framework	Next.js App Router (70%); Vite React SPA + Supabase Edge Functions (20%); SvelteKit (10%)	Server routes keep the Meta key secret; server-rendered storefront gives fast first load and WhatsApp link previews	Next.js App Router
T3	Hosting	Vercel Hobby, Mumbai functions (65%); Cloudflare Workers (20%); Render (15%)	Zero-config Next.js, 300 s function limit covers AI calls, bom1 region next to the database	Vercel, Mumbai functions

#	Decision	Options (recommendation %)	Why the top option	Owner's choice
T4	Owner sign-in	Google sign-in via Supabase Auth (55%); Firebase phone OTP bridged to Supabase (35%); Supabase phone OTP via an Indian SMS provider (10%)	Free, one tap on Android, no SMS rules; Indian SMS OTP needs DLT registration that cannot finish in time	Google sign-in (PRD FR-01 to be updated)
T5	Voice transcription mode	Record, then upload the file (75%); live streaming over WebSocket (25%)	Meta's file endpoint is simpler, retries cleanly and runs on serverless functions	Record, then upload the file
T6	Muse Spark API surface	Responses API (60%); Chat Completions (40%)	Meta's recommended surface, and SAM 3.1 is only reached through it, so one client covers all models	Responses API
T7	Forecasting method	Exponential smoothing + safety stock + festival bump (80%); plain moving average (15%); ask the LLM to forecast (5%)	Transparent, testable numbers from sparse data; the model only explains	Smoothing + safety stock

Follow-ups from these choices: PRD FR-01 now reads Google sign-in instead of phone OTP. The GitHub repo is public so all three teammates can deploy on Vercel Hobby; secrets stay in environment variables only.

Architecture at a glance

Shelf Voice is one Next.js server between three browser surfaces and two providers; only the server talks to Meta, and files skip the server on their way to storage.



system architecture · 3 clients, 1 server, 2 providers

A voice note goes from the owner app straight into Storage, then the server sends it to Voice Transcribe and Muse Spark and writes the confirmed result to Postgres under the owner's RLS policy.

Recommended stack

The recommended stack is one Next.js app on Vercel, one Supabase project in Mumbai, and the Meta Model API called only from the server. Rows marked with a decision ID change if the owner picks another option.

Layer	Choice	Decision	Cost during hackathon
Language	TypeScript (strict) everywhere	Fixed	Free
App framework	Next.js App Router, React Server Components for the storefront	T2	Free
UI	Tailwind CSS + shadcn/ui (Radix primitives), custom tokens from document 06	Fixed	Free

Layer	Choice	Decision	Cost during hackathon
PWA	Web app manifest + service worker for install and offline shell	Fixed (PRD D2)	Free
Server	Next.js route handlers on Vercel functions, region bom1	T3	Hobby plan, free
Database	Supabase Postgres, region ap-south-1 (Mumbai), with Row Level Security	T1	Free plan
Auth	Supabase Auth	T4	Free up to 50,000 monthly users
File storage	Supabase Storage: voice clips, shelf and bill photos, product cut-outs	T1	Free plan
Validation	Zod schemas at every API boundary and for every model output	Fixed	Free
Reasoning model	Muse Spark 1.3 (standard tier, not contributor)	T6	$1.25 / $4.25 per 1M tokens in / out
Speech to text	Muse Voice Transcribe 1.0	T5	About $0.18 per audio hour
Segmentation	SAM 3.1	Fixed (PRD FR-10)	About $2.50 per 1,000 images
Testing	Vitest (unit), Playwright (end-to-end), pgTAP or SQL tests for RLS	Fixed	Free
Observability	Structured logs + an ai_calls table; tool choice in document 05	Fixed	Free

Meta prices are third-party reports of Meta's pricing page; they are re-checked against the Meta dashboard on day 1.

Frontend architecture

One Next.js app serves three surfaces with different needs: a signed-in owner app, a public storefront, and a public review recorder.

Surface	Route	Rendering	Notes
Owner app	/app/*	Client components for recorder, camera and forms; server components for lists	Signed in; installable as a PWA; Hindi or English interface
Storefront	/s/[shopSlug]	Server-rendered, cached, revalidated when the catalog changes	Public; Open Graph tags so a shared link shows a preview card in WhatsApp
Review recorder	/r/[shopSlug]	Small client page, no sign-in	Opened from the counter QR; one big record button

Recording audio:

- Muse Voice Transcribe accepts mono 16-bit PCM WAV at 16 kHz or 24 kHz, while browsers record WebM/Opus by default. The app captures raw audio with the Web Audio API and encodes a 16 kHz WAV in the browser, so the server never converts audio.
- A 16 kHz mono WAV is about 32 KB per second: a 2-minute owner note is about 3.8 MB, a 15-second review about 0.5 MB.
- Vercel functions accept request bodies up to about 4.5 MB, so audio and photos upload straight to Supabase Storage with a signed upload URL; the server receives only the file path.

Photos:

- Shelf and bill photos use the phone camera through a file input with capture set to the rear camera, then are resized in the browser to at most 1600 px JPEG before upload.

State and data:

- Server state through React Server Components and server actions; client state only for recording and review screens. No global store.
- Interface text in a small Hindi/English dictionary; Devanagari renders with Noto Sans Devanagari.

Offline behaviour:

- The service worker caches the app shell. A recording made without signal is kept in IndexedDB and uploaded when the connection returns, which meets the PRD's no-data-loss requirement.

Backend and API architecture

Every AI action follows propose-then-confirm: the server turns speech or a photo into a proposed change set, and nothing is written to the catalog until the owner confirms it.

Endpoints (REST, JSON, prefix /api/v1; all owner routes require a session):

Method and path	Purpose	Calls Meta?
POST /uploads	Signed upload URL for an audio clip or photo	No
POST /proposals/voice	Audio path + context (setup, catalog, sales) to transcript + proposed change set	Voice Transcribe, Muse Spark
POST /proposals/bill	Bill photo path to proposed delivery lines	Muse Spark
POST /proposals/shelf	Shelf photo path to proposed products with cut-out images	SAM 3.1, Muse Spark
POST /proposals/{id}/confirm	Apply an edited proposal; idempotent by proposal ID	No
GET, POST, PATCH, DELETE /products	Direct catalog edits by tap	No
POST /stock-movements	Sale, delivery or count correction	No
GET /restock	Reorder list with quantities and explanations	Muse Spark (explanations, cached)
GET /reviews	Review feed with transcript, sentiment, items, stock-out flag	No
POST /public/shops/{slug}/reviews	Customer review submission; no session; rate limited	Voice Transcribe, Muse Spark (after response)

Method and path	Purpose	Calls Meta?
POST /shops/{id}/publish	Publish or unpublish the storefront	No

Contract rules (from the API design skill):

- One error shape everywhere: `{ error: { code, message, details? } }` with 400, 401, 403, 404, 409, 422, 429 and 500; no stack traces to clients.
- Zod validates every request body, every environment variable, and every model output before use.
- List endpoints paginate (`page` , `pageSize`); responses use camelCase; enums use UPPER_SNAKE.
- The Meta API key, Supabase service key and any SMS keys live only in server environment variables.
- Route handlers stay thin: validate, check ownership, call one service function, return. Business logic lives in `src/server/services` , so it can be unit-tested without HTTP.

AI layer

Three Meta models do six jobs; every job returns schema-validated JSON or tool calls, and code, not the model, owns every number that is stored.

Job	Model	Input	Output	Est. cost per call
Shop setup extraction	Muse Spark 1.3, structured output, low reasoning	Setup transcript	Shop profile + product list; unspoken fields are null	Under $0.01
Catalog and sales commands	Muse Spark 1.3, tool calling	Transcript + the shop's product names and IDs	Tool calls: add_product, update_price, set_stock, record_sale, record_delivery, ask_clarification	Under $0.01

Job	Model	Input	Output	Est. cost per call
Bill reading	Muse Spark 1.3, image input, structured output	Bill photo + product names and IDs	Lines: name, quantity, unit, matched product ID or null	About $0.01
Shelf to products	SAM 3.1, then Muse Spark 1.3 vision	Shelf photo with a text prompt such as "product package"	Boxes and masks, then cut-outs named by Muse Spark	About $0.01
Review analysis	Muse Spark 1.3, structured output, no tools	Review transcript + product names	Sentiment, items mentioned, stock-out items, one-line summary	Under $0.005
Restock explanation	Muse Spark 1.3, structured output	Numbers computed by the forecast	One sentence in Hindi, English or Hinglish; cached per item per day	Under $0.002

Guardrails:

- Numbers: schemas mark price and quantity as null unless spoken or printed; code rejects non-positive or implausible values and asks the owner.
- Product matching: the model may only return product IDs from the list it was given; the server checks each ID belongs to the signed-in owner's shop.
- Untrusted text: customer transcripts are passed as quoted data in the user turn, the review job has no tools, and its output can only fill the schema. Owner standing rules go in the developer message, which Meta recommends for tone, format and rules.
- Speech accuracy: each transcription call passes the shop's product names as keyword hints and Hindi and English as language hints.
- Confirmation: every proposal is shown to the owner before it is written (see Backend).

Integration details:

- One server module (`src/server/ai/meta.ts`) wraps the official OpenAI SDK pointed at https://api.meta.ai/v1. Swapping the base URL to OpenRouter is the fallback if the key fails from India.
- Model ID pinned to muse-spark-1.3; the contributor tier is never used, because it buys its discount with permission to train on prompts.

- Structured output on the Responses API is set through `text.format` with a JSON schema; every result is parsed again with Zod.
- Timeouts of 30 s per call, one retry with backoff on 429 and 5xx, and a clear error state in the app when both fail.

Budget check: about 600 AI calls during development and demos at roughly $0.01 each is about $6, plus transcription and segmentation well under $5, leaving most of the $150 credit as margin.

Voice pipeline

Meta offers two transcription endpoints, and the recommendation (T5) is the file endpoint: record, upload, transcribe, then extract.

Mode	Endpoint	Fits	Cost to us
File (recommended)	POST https://api.meta.ai/v1/asr/transcribe, multipart	A finished recording; serverless functions; clean retries	Simple: one request per clip
Streaming	wss://api.meta.ai/v1/asr/realtime	Live captions while the owner speaks	Needs a long-lived relay server, because the key cannot go to the browser and Vercel functions do not hold WebSockets

Recommended flow for one owner voice note:

1. Browser records mono 16 kHz WAV (up to 2 minutes for setup, 30 seconds for commands) and shows a live level meter.
2. Browser uploads the file to Supabase Storage with a signed URL, then calls POST /proposals/voice with the path.
3. Server downloads the clip, sends it to Muse Voice Transcribe with the shop's product names as keyword hints and Hindi and English as language hints.
4. Server sends the transcript to Muse Spark for extraction or tool calls (see AI layer) and stores the proposal.

5. Browser shows the transcript and the proposed changes; the owner edits and confirms.

Limits from Meta's speech endpoints: WAV only, 16 or 24 kHz mono, recordings up to 10 minutes, no word timestamps or confidence scores. Because there are no confidence scores, the review screen is the accuracy safeguard.

If live captions are wanted later, a small relay (for example on Fly.io or Cloud Run) can forward the WebSocket; it is excluded from the MVP.

Database, storage and auth

Supabase is recommended because Shelf Voice's data is relational and per-shop isolation is easiest to prove with Postgres Row Level Security.

Database options (T1):

Option	Strengths for Shelf Voice	Weaknesses	Free tier
Supabase Postgres, Mumbai (75%)	SQL joins for sales and forecasts; RLS per shop; auth, storage and cron in one project; Mumbai region	Free projects pause after 7 days without activity	500 MB database, 50,000 monthly users, 5 GB egress, 2 projects
Firebase Firestore (15%)	Mature SDKs; phone auth built in	Document model makes sales aggregation awkward; per-read billing; Spark limits tightened in 2026	Daily read and write quotas
Neon Postgres + own auth (10%)	Pure Postgres with branching	Auth, storage and policies all built by us	Generous compute hours

Core entities (full schema, constraints and indexes in document 07): shops, products, stock_movements (sales, deliveries, corrections), proposals, reviews, ai_calls, and festival_calendar.

Access rules:

- Every owner table carries shop_id; RLS policies allow access only when the shop's owner_id equals the signed-in user.

- Public storefront and review pages read through a narrow view of published shops and in-stock products, never the base tables.
- The service-role key is used only by server code for public review inserts and background jobs, and never reaches the browser.

Storage buckets: `voice` (private), `photos` (private), `catalog-images` (public product cut-outs). Clips older than 30 days are deleted (retention in document 03).

Owner sign-in options (T4):

Option	Fit for kirana owners	Blocker for the hackathon	Cost
Google sign-in via Supabase Auth (55%)	Every Android phone has a Google account; one tap	None; changes PRD FR-01	Free
Firebase phone OTP bridged to Supabase (35%)	Most familiar for Indian users	Second vendor; Firebase billing account; tokens bridged through Supabase third-party auth	About $0.07 per SMS in India after 10 free a day
Supabase phone OTP via MSG91 or Twilio (10%)	Cheapest SMS at scale (about Rs 0.20 each)	India requires DLT registration of entity, sender ID and templates before any SMS delivers; not possible by Nov 4	Rs 5,900 DLT fee + per SMS

Whatever the choice, judges get a one-tap "Try the demo shop" sign-in to a seeded account, so the demo never depends on SMS delivery.

Risk: the free Supabase project pauses after 7 days of inactivity, which could hit during judging (Nov 5 to 9). Mitigation: a daily scheduled health check keeps it active, or upgrade to Pro ($25) for judging week.

Restock forecasting method

Forecasts are computed in SQL and TypeScript with a standard inventory formula, so every suggested quantity can be checked by hand; Muse Spark only turns the numbers into a sentence.

Per product, each night and after every logged sale:

1. Daily demand level: exponential smoothing over daily units sold, days with no sales counted as zero, smoothing factor 0.3.
2. Adjustments: multiply by the festival factor when a listed festival is within 7 days (default 1.0), and by 1.2 for 14 days after a customer review flags the item as out of stock, since stock-outs hide real demand.
3. Safety stock, reorder point and order quantity, with lead time L days (owner sets, default 2), review period R days (default 7), z = 1.28 for a 90% service level, and σ the standard deviation of daily sales:

```
\begin{aligned}
d &= \text{level} \times f_{\text{festival}} \times f_{\text{stockout}} \\
SS &= z \, \sigma \sqrt{L} \\
ROP &= d \, L + SS \\
Q &= \left\lceil d\,(L + R) + SS - \text{stock} \right\rceil_{\text{pack
size}} \\
\text{days left} &= \text{stock} / d
\end{aligned}
```

4. An item enters the reorder list when stock is at or below ROP; the list is sorted by days left, with stock-out-flagged items first.

Cold start: with fewer than 14 days of sales, the app uses the owner's own low-stock level per item (default 5 units) and says that forecasts start after two weeks of logging.

Why not the alternatives: a plain moving average reacts slowly to trend changes; asking the LLM for quantities makes numbers untestable and can invent them. Intermittent-demand methods such as Croston's are a later improvement for slow sellers.

All factors (0.3, 1.2, 1.28, defaults) are configuration values, tuned in the backtest defined in document 04.

Background jobs, caching and performance

The MVP needs no queue service: slow work runs after the HTTP response or on a daily schedule.

Job	Trigger	Runs on	Failure handling
Review analysis	Customer submits a review	Next.js `after()` on the same function, so the customer gets a thank-you at once	Review saved as PENDING; the nightly job retries anything still pending

Job	Trigger	Runs on	Failure handling
Forecast refresh	After each logged sale, and nightly	Postgres function, scheduled with pg_cron	Idempotent; recomputes from raw movements
Restock explanations	When an item first enters the reorder list that day	Server, on request	Falls back to a template sentence built from the numbers
Clip cleanup and keep-alive	Daily	Vercel cron hitting a protected route	Logged; a missed run is harmless

Caching:

- Storefront pages are statically cached and revalidated by tag when that shop's catalog changes, so most customer visits never touch the database.
- Restock explanations are stored per item per day; Meta's prompt caching reduces the cost of the repeated developer message.

Performance budget: storefront first load under 3 s on 4G (JavaScript under 100 KB for that page), voice note to proposal under 25 s end to end, all list queries indexed on shop_id plus date.

Deployment model and environments

Vercel is recommended for hosting (T3), with functions pinned to Mumbai (bom1) beside the Supabase database in ap-south-1.

Option	Strengths	Weaknesses	Free tier
Vercel Hobby (65%)	Built for Next.js; preview URL per pull request; 300 s function limit; Mumbai region; 1 million invocations a month	Hobby is one seat and non-commercial; teammates' commits to a private repo may not auto-deploy, so the repo is public (decided Oct 27)	Free
Cloudflare Workers (20%)	Generous free tier, unlimited seats, global edge	Next.js runs through the OpenNext adapter; more setup and fewer Node APIs	Free
Render (15%)	Plain Node server, WebSockets possible	Free services sleep when idle, giving slow first loads in a live demo	Free with sleep

Environments:

Environment	App	Database	Purpose
Local	`next dev`	Supabase CLI local stack (Docker)	Feature work and tests; migrations applied from `supabase/migrations`
Preview	Vercel preview per pull request	Staging Supabase project (free plan allows 2)	Review and end-to-end tests
Production	Vercel production, custom domain optional	Production Supabase project	Demo, judges, real shop owners

Configuration: all settings come from environment variables validated with Zod at startup; `.env.example` lists every variable with placeholders. Secrets are set in Vercel and Supabase dashboards, never committed. The full runbook is document 08.

Architecture decision records

Each record becomes final when its decision in the table at the top is confirmed; reversal paths are noted so no choice traps the project.

ADR	Decision	Context	Consequence	Reversal path
ADR-001	Propose-then-confirm for every AI write	Speech and photos are error-prone and transcripts carry no confidence scores	One extra tap per action; no silent data corruption	None needed
ADR-002	Numbers computed in code, never by the model	Forecast quantities and prices must be testable and trusted	Forecast logic is ordinary, unit-tested code	Swap formula without touching AI prompts
ADR-003	Server-only access to the Meta API	Keys must never reach the browser	All AI goes through route handlers	None needed
ADR-004	Supabase Postgres with RLS as system of record (T1)	Relational data, per-shop isolation, one vendor for DB, auth, storage	Free project pauses when idle; mitigated by keep-alive	Plain Postgres dump restores to any Postgres host
ADR-005	File-based transcription, not streaming (T5)	Serverless hosting, retries, simplicity	No live captions while speaking	Add a WebSocket relay later
ADR-006	Audio and photos upload directly to storage	Function request bodies cap at about 4.5 MB	Two-step upload in the client	None needed
ADR-007	One AI client module, OpenAI-compatible	Possible regional access problem; OpenRouter also serves the Muse models	Fallback is a base-URL change	Point the module at another provider
ADR-008	Exponential smoothing with safety stock (T7)	Sparse per-shop data; judges value sound engineering	Transparent and testable; not a learned model	Replace with Croston or a learned model per product later

Sources

Checked Oct 27, 2026. Items marked third-party are to be re-confirmed in Meta's own dashboard on day 1.

- Meta Model Cookbook: Muse Voice API fundamentals: file and realtime transcription endpoints
- Muse Voice Transcribe on OpenRouter: WAV 16/24 kHz mono, up to 10 minutes, no timestamps or confidence (third-party)
- Pipecat Meta STT reference: realtime endpoint, accepted sample rates, language biasing
- SAM 3.1 via Meta Model API: model sam-3.1 on the Responses API, about $2.50 per 1,000 images (third-party)
- SAM 3.1 demo client: request shape on /v1/responses
- Muse Spark 1.3 on OpenRouter: structured output and tool calling; contributor tier pricing (third-party)
- Muse Spark pricing and rate limits: standard rate, per-team limits, 1.3 recommended for new work (third-party)
- promptfoo Meta provider: Responses as default surface, rate limits per team
- Muse Wonder API guide: structured output on Responses uses text.format
- Supabase pricing 2026: free tier limits and 7-day pause (third-party)
- Supabase Firebase Auth integration: third-party auth with RLS
- Firebase pricing 2026: phone sign-in about $0.07 per SMS to India (third-party)
- DLT registration in India: SMS registration steps and fee
- Vercel Hobby plan: 300 s functions, non-commercial use
- Vercel fluid compute pricing: Mumbai bom1 region